

Assignment 3: RNN for Text Generation

Title

Character-Level Recurrent Neural Network for Text Generation

Problem Statement

Implement a character-level RNN (with LSTM/GRU cells) to learn patterns in sequential text data and generate new text that mimics the style and structure of the training corpus. You will build the model from scratch or using PyTorch/Keras, train it on a literary dataset, and generate creative text samples.

Dataset

Shakespeare Text Corpus

- **Source:** [Shakespeare Complete Works](#)
- **Alternative:** [Project Gutenberg](#) - choose any classic book
- **Description:** ~1MB of Shakespeare's plays and sonnets, character-level modelling

Detailed Instructions

Part 1: Text Preprocessing (15%)

1. Download and load the text corpus
2. Perform basic text analysis:
 - Total characters, unique characters
 - Character frequency distribution
 - Vocabulary size
3. Create character-to-index and index-to-character mappings
4. Generate training sequences:
 - Sequence length: 100 characters
 - Use sliding window approach (stride=3)
 - Create input (X) and target (y) pairs where $y = X$ shifted by 1
5. Encode characters as one-hot vectors or integer indices
6. Split into training (90%) and validation (10%) sets

Part 2: RNN Model Architecture (35%)

Task 2.1: Implement Basic RNN Build a simple RNN with:

Embedding(vocab_size, 128) →

LSTM(256, return_sequences=True) →

Dropout(0.3) →

LSTM(256) →

Dense(vocab_size, activation='softmax')

Task 2.2: Advanced RNN Options Choose one enhancement:

- **Option A:** Multi-layer LSTM (3 layers with decreasing units: 512→256→128)
- **Option B:** Bidirectional LSTM for better context understanding
- **Option C:** GRU cells instead of LSTM (compare performance)
- **Option D:** Attention mechanism on top of LSTM

Task 2.3: Model Configuration

- Use embedding dimension: 128 or 256
- Hidden state size: 256 or 512
- Number of layers: 2-3
- Dropout: 0.2-0.4 between layers

Part 3: Training Procedure (25%)

1. **Loss Function:** Categorical cross-entropy (for character prediction)
2. **Optimizer:** Adam with learning rate = 0.001
3. **Training Strategy:**
 - Epochs: 30-50
 - Batch size: 128
 - Gradient clipping (max_norm=5) to prevent exploding gradients
 - Model checkpointing (save every 5 epochs)
4. **Monitoring:**
 - Track training and validation loss
 - Generate sample text every 5 epochs
 - Monitor perplexity metric

Part 4: Text Generation (15%)

1. Implement Sampling Function:

- Seed text: Start with a random sequence from the dataset
- Temperature-based sampling: Implement temperature parameter (0.2, 0.5, 1.0, 1.2)
- Top-k sampling (k=5, k=10)
- Top-p (nucleus) sampling (p=0.9)

2. Generate Text Samples:

- Generate 500-character sequences with different temperatures
- Use various seed texts (5-10 different starting phrases)
- Compare sampling strategies

3. Interactive Generation:

- Allow user to input custom seed text
- Generate continuation of specified length

Part 5: Analysis & Experimentation (10%)

1. Compare LSTM vs. GRU performance
2. Analyze the effect of sequence length (50, 100, 200)
3. Study temperature effect on text diversity vs. coherence
4. Visualize hidden state activations using t-SNE
5. Analyze which character combinations the model learns best

Expected Outputs

1. Code Deliverables

- Data preprocessing script
- Model training script
- Text generation script with sampling functions
- Jupyter notebook with experiments

2. Model Files

- Trained model weights (checkpoints at epochs 10, 20, 30, final)
- Vocabulary mappings (char2idx, idx2char) as JSON
- Training history logs

3. **Generated Text Samples**

- 10 samples with temperature = 0.5 (coherent)
- 10 samples with temperature = 1.0 (balanced)
- 10 samples with temperature = 1.2 (creative)
- Comparison of different sampling strategies
- Save all samples in a text file

4. **Visualizations**

- Training/validation loss curves
- Perplexity over epochs
- Character frequency distribution (original vs. generated)
- Hidden state t-SNE visualization
- Temperature effect comparison (coherence vs. diversity plot)

5. **Analysis Report** (3-4 pages)

- Architecture design decisions
- Training challenges and solutions (gradient issues, overfitting)
- Qualitative analysis of generated text
- Temperature parameter effects
- Comparison of LSTM vs. GRU (if implemented)
- Limitations and potential improvements

Findings/Observations to Document

1. How does sequence length affect learning quality and training time?
2. At what epoch does the model start generating coherent text?
3. Which temperature setting produces the best balance of creativity and coherence?
4. What patterns (words, phrases) does the model learn first?
5. How do different sampling strategies (greedy, temperature, top-k, top-p) compare?
6. What types of errors does the model make most frequently?
7. Can the model learn long-range dependencies (e.g., matching opening/closing quotes)?

Evaluation Rubric (100 points)

Criteria	Points	Description
Code Implementation	35	Correct RNN architecture, proper sequence generation, sampling functions
Model Training	20	Successful training, proper gradient handling, convergence achieved
Text Quality	20	Generated text shows learned patterns, reasonable coherence, vocabulary usage
Experimentation	10	Temperature comparison, sampling strategies, architecture variations
Visualizations	5	Loss curves, character distributions, hidden state analysis
Report & Analysis	10	Insightful observations, qualitative evaluation, improvement suggestions

Bonus Points (10)

- Implement attention mechanism (+3)
- Word-level model comparison (+2)
- Real-time generation web interface (+3)
- Generate poetry with proper rhyme scheme (+2)

Target Metrics

- **Validation Loss:** <1.5 (good), <1.2 (excellent)
- **Perplexity:** <5 (good), <4 (excellent)
- **Text Quality:** Qualitative assessment by peers/instructor
